Projet UL2IN002 – 2026 fev Thème : “la Mer”

Numéro de groupe : Mono 5

Nom : Nasimi	*Nom :*
Prénom : Mohamad Abobaker	*Prénom :*
N° étudiant : 21414201	*N° étudiant :*

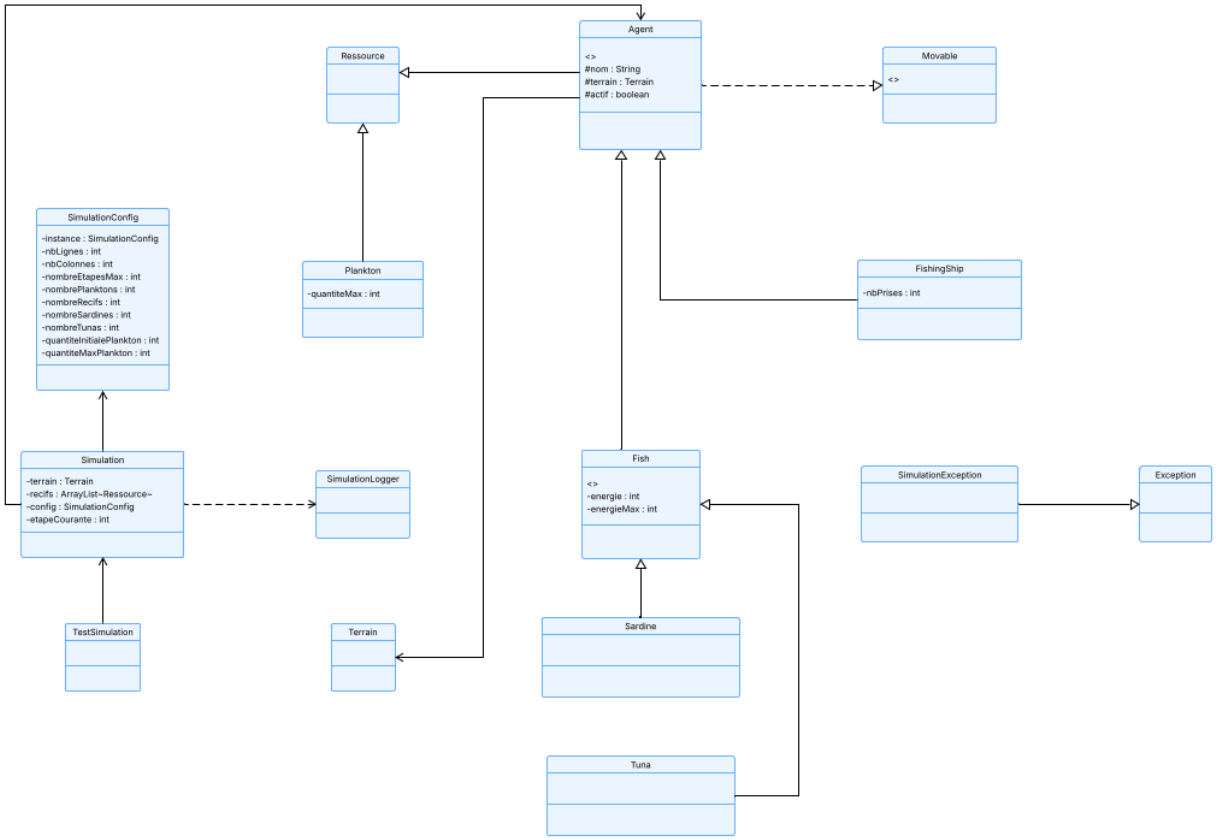
	Thème de la simulation
Thème et objectif(s) de votre simultation : (5 lignes maximum)	Cette simulation représente une chaîne alimentaire marine simple. Le plancton est une ressource présente dans la mer et peut grandir au fil du temps. Les sardines se déplacent sur le terrain pour chercher et manger le plancton. Les thons se déplacent pour chasser les sardines. Enfin, un bateau de pêche se déplace pour capturer les thons.
Les agents :	Les agents sont les sardines, les thons et le bateau de pêche.
Les ressources :	Les ressources sont le plancton (ressource dynamique) et les récifs (ressource fixe).

Quel est le nombre de classes/interfaces de votre projet ? (ne pas compter Ressource, Terrain et TestTerrain)	12
Avez-vous une hiérarchie d'héritage à 3 niveaux ?	Oui
Avez-vous généré la javadoc ?	Oui
Avez-vous généré des logs ?	Oui

Checklist des points vus en cours à mettre en application :	***Nom de classe correspondant (une seule classe demandée, mais vous pouvez en mettre plus)***
Classe contenant une ArrayList	Simulation
Classe abstraite et méthode abstraite	Agent
Interface	Movable
Classe avec un constructeur par copie ou méthoe clone()	Plankton

Classe contenant que des attributs et méthodes statiques	SimulationLogger
Classe héritant de Exception	SimulationException
Gestion des exceptions	Simulation et TestSimulation
Singleton ou une de ses variantes	SimulationConfig

Schéma UML des classes vision fournisseur (dessin “à la main” scanné ou photo acceptés)



Copier-coller de 2 ou 3 logs (affichages de simulations)

```
-----------------------------------------------
Simulation initialisee.
Resume: planctons=15, recifs=6, sardines=6, tunas=3
Sardine-1 en (5,1)
Sardine-2 en (10,7)
Sardine-3 en (8,8)
Sardine-4 en (8,4)
Sardine-5 en (5,4)
Sardine-6 en (3,5)
Tuna-1 en (3,1)
Tuna-2 en (7,5)
Tuna-3 en (4,4)
Ship-1 en (8,6)
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |   |   |   |   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |Pla|   |   |   |Pla|   |   |
:---:---:---:---:---:---:---:---:---:---:
|Tun|   |   |Pla|Sar|   |   |   |Pla|   |
```

:---:---:---:---:---:---:---:---:---:---:
| | | |Tun|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Sar|Rec| |Sar|Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec|Tun| |Pla| | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | |Sar| |Shi| |Sar| | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| |Pla| |Sar| | | |
:---:---:---:---:---:---:---:---:---:---:

Debut de la simulation marine

ETAPE 1
Plancton: 15 ressource(s) ont grandi.
Sardine-1 se deplace vers le plancton.
Sardine-2 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (9,4).
Sardine-5 mange un peu de plancton en (4,4).
Sardine-6 mange un peu de plancton en (3,5).
Tuna-1 mange Sardine-1.
Tuna-2 se deplace vers une sardine.
Tuna-3 mange Sardine-5.
Ship-1 capture Tuna-2 (prises=1).
Resume: planctons=15, recifs=6, sardines=4, tunas=2
Sardine-2 en (10,6)
Sardine-3 en (7,8)
Sardine-4 en (8,4)
Sardine-6 en (3,5)
Tuna-1 en (4,1)
Tuna-3 en (5,4)
Ship-1 en (8,5)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Sar| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
|Tun| | |Pla|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| |Tun|Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | | |

:---:---:---:---:---:---:---:---:---:---:
| | | |Rec| | |Pla|Sar| |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | |Sar|Shi|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| |Pla|Sar| | | | |
:---:---:---:---:---:---:---:---:---:---:

ETAPE 2
Plancton: 15 ressource(s) ont grandi.
Sardine-2 mange un peu de plancton en (10,5).
Sardine-3 mange un peu de plancton en (7,7).
Sardine-4 mange un peu de plancton en (9,4).
Sardine-6 mange un peu de plancton en (3,5).
Tuna-1 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=15, recifs=6, sardines=4, tunas=2
Sardine-2 en (10,6)
Sardine-3 en (7,8)
Sardine-4 en (8,4)
Sardine-6 en (3,5)
Tuna-1 en (3,1)
Tuna-3 en (6,4)
Ship-1 en (7,5)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
|Tun| | |Pla|Sar| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| | |Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Tun| | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec|Shi| |Pla|Sar| |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | |Sar| |Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| |Pla|Sar| | | | |
:---:---:---:---:---:---:---:---:---:---:

ETAPE 3

Plancton: 15 ressource(s) ont grandi.
Sardine-2 mange un peu de plancton en (10,5).
Sardine-3 mange un peu de plancton en (7,7).
Sardine-4 mange tout le plancton en (9,4).
Sardine-6 mange tout le plancton en (3,5).
Tuna-1 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 capture Tuna-3 (prises=2).
Resume: planctons=13, recifs=6, sardines=4, tunas=1
Sardine-2 en (10,6)
Sardine-3 en (7,8)
Sardine-4 en (8,4)
Sardine-6 en (3,5)
Tuna-1 en (3,2)
Ship-1 en (7,4)

			Pla				Pla		
	Tun		Pla	Sar			Pla		
			Pla	Pla			Pla	Rec	
Rec	Rec		Rec	Pla					
			Shi		Pla	Sar		Pla	
			Sar		Pla				
							Rec		
		Pla		Pla	Sar				

ETAPE 4
Plancton: 3 ressource(s) ont grandi.
Sardine-2 mange un peu de plancton en (10,5).
Sardine-3 mange un peu de plancton en (7,7).
Sardine-4 se deplace vers le plancton.
Sardine-6 mange un peu de plancton en (4,5).
Tuna-1 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=13, recifs=6, sardines=4, tunas=1
Sardine-2 en (10,6)
Sardine-3 en (7,8)
Sardine-4 en (8,5)
Sardine-6 en (3,5)

Tuna-1 en (3,3)
Ship-1 en (6,4)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Tun|Pla|Sar| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| | |Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Shi| | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec| | |Pla|Sar| |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | |Sar|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| |Pla|Sar| | | | |
:---:---:---:---:---:---:---:---:---:---:

ETAPE 5
Plancton: 4 ressource(s) ont grandi.
Sardine-2 mange tout le plancton en (10,5).
Sardine-3 mange tout le plancton en (7,7).
Sardine-4 mange un peu de plancton en (8,6).
Sardine-6 mange un peu de plancton en (4,5).
Tuna-1 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=11, recifs=6, sardines=4, tunas=1
Sardine-2 en (10,6)
Sardine-3 en (7,8)
Sardine-4 en (8,5)
Sardine-6 en (3,5)
Tuna-1 en (3,4)
Ship-1 en (5,4)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Tun|Sar| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| |Shi|Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:

| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec| | | |Sar| |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | |Sar|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| | |Sar| | | | |
:---:---:---:---:---:---:---:---:---:---:

ETAPE 6
Plancton: 2 ressource(s) ont grandi.
Sardine-2 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (8,6).
Sardine-6 mange un peu de plancton en (4,5).
Tuna-1 mange Sardine-6.
Ship-1 se deplace vers un thon.
Resume: planctons=11, recifs=6, sardines=3, tunas=1
Sardine-2 en (9,6)
Sardine-3 en (7,9)
Sardine-4 en (8,5)
Tuna-1 en (3,5)
Ship-1 en (4,4)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Tun| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Shi|Pla| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| | |Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec| | | | |Sar|Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | |Sar|Pla| | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Sar| |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| | | | | | | |
:---:---:---:---:---:---:---:---:---:---:

ETAPE 7
Plancton: 2 ressource(s) ont grandi.
Sardine-2 mange un peu de plancton en (8,6).

Sardine-3 mange un peu de plancton en (7,10).
Sardine-4 mange tout le plancton en (8,6).
Tuna-1 se deplace vers une sardine.
Ship-1 capture Tuna-1 (prises=3).
Resume: planctons=10, recifs=6, sardines=3, tunas=0
Sardine-2 en (9,6)
Sardine-3 en (7,9)
Sardine-4 en (8,5)
Ship-1 en (4,5)
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla| | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Shi| | | |Pla|Rec|
:---:---:---:---:---:---:---:---:---:---:
|Rec|Rec| | |Rec|Pla| | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Rec| | | | |Sar|Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | |Sar| | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | |Sar| |Rec| | |
:---:---:---:---:---:---:---:---:---:---:
| | |Pla| | | | | | | |
:---:---:---:---:---:---:---:---:---:---:
Arret: il n'y a plus de thons.

Fin de la simulation.
Prises du bateau : 3

Simulation initialisee.
Resume: planctons=15, recifs=6, sardines=6, tunas=3
Sardine-1 en (2,5)
Sardine-2 en (5,3)
Sardine-3 en (5,2)
Sardine-4 en (8,2)
Sardine-5 en (3,8)
Sardine-6 en (2,2)
Tuna-1 en (9,6)
Tuna-2 en (6,7)
Tuna-3 en (4,8)
Ship-1 en (2,4)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | | |Pla| | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

| |Sar|Pla|Shi|Sar| | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Rec|Sar| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | |Tun|Pla| |
:---:---:---:---:---:---:---:---:---:---:
| |Sar|Sar| | |Pla|Pla| | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla| |Tun|Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
|Pla| | | |Tun| | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

Debut de la simulation marine

ETAPE 1
Plancton: 15 ressource(s) ont grandi.
Sardine-1 mange un peu de plancton en (1,5).
Sardine-2 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (8,2).
Sardine-5 se deplace vers le plancton.
Sardine-6 mange un peu de plancton en (2,3).
Tuna-1 se deplace vers une sardine.
Tuna-2 se deplace vers une sardine.
Tuna-3 mange Sardine-5.
Ship-1 se deplace vers un thon.
Resume: planctons=15, recifs=6, sardines=5, tunas=3
Sardine-1 en (2,5)
Sardine-2 en (5,4)
Sardine-3 en (6,2)
Sardine-4 en (8,2)
Sardine-6 en (2,2)
Tuna-1 en (8,6)
Tuna-2 en (5,7)
Tuna-3 en (4,8)
Ship-1 en (3,4)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | |Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Sar|Pla| |Sar| | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | |Shi| |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:

| | | | | | | |Tun|Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Sar| |Pla|Tun| | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Pla| | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Pla|Tun| | | | |
:---:---:---:---:---:---:---:---:---:---:
|Pla| | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

ETAPE 2
Plancton: 15 ressource(s) ont grandi.
Sardine-1 mange un peu de plancton en (1,5).
Sardine-2 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (8,2).
Sardine-6 mange un peu de plancton en (2,3).
Tuna-1 se deplace vers une sardine.
Tuna-2 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=15, recifs=6, sardines=5, tunas=3
Sardine-1 en (2,5)
Sardine-2 en (5,5)
Sardine-3 en (7,2)
Sardine-4 en (8,2)
Sardine-6 en (2,2)
Tuna-1 en (7,6)
Tuna-2 en (5,6)
Tuna-3 en (5,8)
Ship-1 en (4,4)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | |Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Sar|Pla| |Sar| | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:
| | | |Shi| | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | |Sar|Tun|Pla|Tun| |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla| | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | | |Tun| | |Pla| |

:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
|Pla| | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

ETAPE 3
Plancton: 15 ressource(s) ont grandi.
Sardine-1 mange tout le plancton en (1,5).
Sardine-2 mange un peu de plancton en (5,6).
Sardine-3 mange tout le plancton en (8,2).
Sardine-4 se deplace vers le plancton.
Sardine-6 mange tout le plancton en (2,3).
Tuna-1 se deplace vers une sardine.
Tuna-2 mange Sardine-2.
Tuna-3 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=12, recifs=6, sardines=4, tunas=3
Sardine-1 en (2,5)
Sardine-3 en (7,2)
Sardine-4 en (9,2)
Sardine-6 en (2,2)
Tuna-1 en (6,6)
Tuna-2 en (5,5)
Tuna-3 en (4,8)
Ship-1 en (5,4)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | | | | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Sar| | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | |Tun|Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Shi|Tun|Pla|Pla| | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla|Tun| |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
|Pla|Sar| | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

ETAPE 4
Plancton: 1 ressource(s) ont grandi.
Sardine-1 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (9,1).
Sardine-6 se deplace vers le plancton.
Tuna-1 se deplace vers une sardine.
Tuna-2 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=12, recifs=6, sardines=4, tunas=3
Sardine-1 en (3,5)
Sardine-3 en (8,2)
Sardine-4 en (9,2)
Sardine-6 en (3,2)
Tuna-1 en (5,6)
Tuna-2 en (4,5)
Tuna-3 en (3,8)
Ship-1 en (5,5)

Rec									Rec
									Pla
	Sar		Sar		Rec	Tun			
				Tun			Pla		
				Shi	Tun	Pla		Rec	
	Rec		Pla		Pla				
							Pla		
	Sar		Pla						
Pla	Sar						Pla		
			Pla	Pla				Rec	

ETAPE 5
Plancton: 2 ressource(s) ont grandi.
Sardine-1 se deplace vers le plancton.
Sardine-3 se deplace vers le plancton.
Sardine-4 mange un peu de plancton en (9,1).
Sardine-6 se deplace vers le plancton.
Tuna-1 se deplace vers une sardine.
Tuna-2 mange Sardine-1.
Tuna-3 se deplace vers une sardine.

Ship-1 capture Tuna-2 (prises=1).
Resume: planctons=12, recifs=6, sardines=3, tunas=2
Sardine-3 en (9,2)
Sardine-4 en (9,2)
Sardine-6 en (4,2)
Tuna-1 en (4,6)
Tuna-3 en (4,8)
Ship-1 en (4,5)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | | | | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Shi|Tun| |Tun|Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | | |Pla|Pla| | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla| | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
|Pla|Sar| | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

ETAPE 6
Plancton: 1 ressource(s) ont grandi.
Sardine-3 mange un peu de plancton en (9,1).
Sardine-4 mange tout le plancton en (9,1).
Sardine-6 se deplace vers le plancton.
Tuna-1 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 se deplace vers un thon.
Resume: planctons=11, recifs=6, sardines=3, tunas=2
Sardine-3 en (9,2)
Sardine-4 en (9,2)
Sardine-6 en (5,2)
Tuna-1 en (5,6)
Tuna-3 en (5,8)
Ship-1 en (5,5)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | | | | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | |Pla|
:---:---:---:---:---:---:---:---:---:---:

| | | | | | |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | |Shi|Tun|Pla|Tun| |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla| | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
| |Sar| | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | |Pla|Pla| | | |Rec|
:---:---:---:---:---:---:---:---:---:---:

ETAPE 7
Plancton: 0 ressource(s) ont grandi.
Sardine-3 se deplace vers le plancton.
Sardine-4 se deplace vers le plancton.
Sardine-6 se deplace vers le plancton.
Tuna-1 se deplace vers une sardine.
Tuna-3 se deplace vers une sardine.
Ship-1 capture Tuna-1 (prises=2).
Resume: planctons=11, recifs=6, sardines=3, tunas=1
Sardine-3 en (10,2)
Sardine-4 en (10,2)
Sardine-6 en (5,3)
Tuna-3 en (5,7)
Ship-1 en (5,5)
:---:---:---:---:---:---:---:---:---:---:
|Rec| | | | | | | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | | |Pla|
:---:---:---:---:---:---:---:---:---:---:
| | | | | | |Rec| | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | |Sar| |Shi|Pla|Tun| | |Rec|
:---:---:---:---:---:---:---:---:---:---:
| |Rec| | |Pla| | |Pla| | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:
| | | | |Pla| | | | | |
:---:---:---:---:---:---:---:---:---:---:
| | | | | | | | |Pla| |
:---:---:---:---:---:---:---:---:---:---:

```
|   |Sar|   |Pla|Pla|   |   |   |   |Rec|
:---:---:---:---:---:---:---:---:---:---:
-----------------------------------------------
ETAPE 8
Plancton: 0 ressource(s) ont grandi.
Sardine-3 se deplace vers le plancton.
Sardine-4 se deplace vers le plancton.
Sardine-6 se deplace vers le plancton.
Tuna-3 se deplace vers une sardine.
Ship-1 capture Tuna-3 (prises=3).
Resume: planctons=11, recifs=6, sardines=3, tunas=0
Sardine-3 en (10,3)
Sardine-4 en (10,3)
Sardine-6 en (5,4)
Ship-1 en (5,6)
:---:---:---:---:---:---:---:---:---:---:
|Rec|   |   |   |   |   |   |   |Rec|
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |   |   |   |Pla|
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |Rec|   |   |   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |   |   |Pla|   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |Sar|   |Shi|Pla|   |   |Rec|
:---:---:---:---:---:---:---:---:---:---:
|   |Rec|   |   |Pla|   |   |Pla|   |   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |   |   |Pla|   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |Pla|   |   |   |   |   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |   |   |   |   |   |   |Pla|   |
:---:---:---:---:---:---:---:---:---:---:
|   |   |Sar|Pla|Pla|   |   |   |   |Rec|
:---:---:---:---:---:---:---:---:---:---:
Arret: il n'y a plus de thons.
-----------------------------------------------
Fin de la simulation.
Prises du bateau : 3
```

Copier-coller vos classes et interfaces à partir d'ici :

```
/*****************************************************************************/
/*                                                                           */
/*                          Sorbonne Universite                              */
/*                     UL2IN002 - Programmation Objet                        */
/*                                                                           */
```

```
/*                          File: Agent.java                          */
/*                  Etudiant: Mohamad Abobaker NASIMI                  */
/*                                                                     */
/*******************************************************************************/

/**
 * Classe de base pour tous les agents mobiles de la simulation.
 * On herite de Ressource pour reutiliser simplement la gestion de position (ligne,
colonne).
 */
public abstract class Agent extends Ressource implements Movable {
    protected String nom;
    protected Terrain terrain;
    protected boolean actif;

    public Agent(String type, String nom, Terrain terrain) {
        super(type, 1);
        this.nom = nom;
        this.terrain = terrain;
        this.actif = true;
    }

    public String getNom() {
        return nom;
    }

    public boolean estActif() {
        return actif;
    }

    public void desactiver() {
        this.actif = false;
    }

    public int distanceA(int ligne, int colonne) {
        if (!estPlace()) {
            return Integer.MAX_VALUE;
        }
        return Math.abs(getLigne() - ligne) + Math.abs(getColonne() - colonne);
    }

    public int distanceA(Ressource ressource) {
        if (ressource == null || ressource.getLigne() == -1 || ressource.getColonne() == -1) {
            return Integer.MAX_VALUE;
        }
        return distanceA(ressource.getLigne(), ressource.getColonne());
    }

    public boolean estPlace() {
        return getLigne() != -1 && getColonne() != -1;
```

```
    }

    public void retirerDuTerrain() {
        // Les agents ne sont pas stockes dans Terrain : on vide juste leur position interne.
        resetPosition();
    }

    protected void desactiverEtRetirer() {
        desactiver();
        retirerDuTerrain();
    }

    /**
     * Action principale executee a chaque etape.
     */
    public abstract String agir(Simulation simulation) throws SimulationException;

    @Override
    public boolean seDeplacer(int ligneCible, int colonneCible) throws SimulationException {
        if (!actif || !estPlace()) {
            return false;
        }

        if (!terrain.sontValides(ligneCible, colonneCible)) {
            throw new SimulationException("Cible invalide pour " + nom + " : (" + ligneCible +
"," + colonneCible + ")");
        }

        int ligneActuelle = getLigne();
        int colonneActuelle = getColonne();

        int nouvelleLigne = ligneActuelle;
        int nouvelleColonne = colonneActuelle;

        if (ligneCible > ligneActuelle) {
            nouvelleLigne++;
        } else if (ligneCible < ligneActuelle) {
            nouvelleLigne--;
        } else if (colonneCible > colonneActuelle) {
            nouvelleColonne++;
        } else if (colonneCible < colonneActuelle) {
            nouvelleColonne--;
        }

        if (nouvelleLigne == ligneActuelle && nouvelleColonne == colonneActuelle) {
            return false;
        }

        if (!terrain.sontValides(nouvelleLigne, nouvelleColonne)) {
            return false;
```

```
        }

        setPosition(nouvelleLigne, nouvelleColonne);
        return true;
    }
}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                        File: FishingShip.java                             */
/*                   Etudiant: Mohamad Abobaker NASIMI                       */
/*                                                                           */
/*****************************************************************************/

import java.util.ArrayList;

/**
 * Bateau de peche : il cherche les thons et les capture.
 */
public class FishingShip extends Agent {
    private int nbPrises;

    public FishingShip(String nom, Terrain terrain) {
        super("Ship", nom, terrain);
        this.nbPrises = 0;
    }

    @Override
    public String agir(Simulation simulation) throws SimulationException {
        return agirAvecTunas(simulation.getTunas());
    }

    public int getNbPrises() {
        return nbPrises;
    }

    private String agirAvecTunas(ArrayList<Tuna> tunas) throws SimulationException {
        if (!estActif()) {
            return nom + " est inactif.";
        }

        Tuna cible = chercherTunaLePlusProche(tunas);
        if (cible == null) {
            return nom + " ne trouve plus de thon.";
        }

        int distance = distanceA(cible);
        int ligCible = cible.getLigne();
```

```
        int colCible = cible.getColonne();

        if (distance <= 1) {
            cible.desactiverEtRetirer();
            tunas.remove(cible);
            nbPrises++;

            seDeplacer(ligCible, colCible);
            return nom + " capture " + cible.getNom() + " (prises=" + nbPrises + ").";
        }

        boolean deplace = seDeplacer(ligCible, colCible);
        if (deplace) {
            return nom + " se deplace vers un thon.";
        }
        return nom + " est bloque.";
    }

    private Tuna chercherTunaLePlusProche(ArrayList<Tuna> tunas) {
        Tuna meilleureCible = null;
        int meilleureDistance = Integer.MAX_VALUE;

        for (Tuna t : tunas) {
            if (t == null || !t.estActif() || t.getLigne() == -1 || t.getColonne() == -1) {
                continue;
            }
            int d = distanceA(t);
            if (d < meilleureDistance) {
                meilleureDistance = d;
                meilleureCible = t;
            }
        }
        return meilleureCible;
    }
}
/*******************************************************************************/
/*                                                                             */
/*                        Sorbonne Universite                                  */
/*                    UL2IN002 - Programmation Objet                           */
/*                                                                             */
/*                         File: Fish.java                                     */
/*                  Etudiant: Mohamad Abobaker NASIMI                          */
/*                                                                             */
/*******************************************************************************/

public abstract class Fish extends Agent {
    private int energie;
    private int energieMax;

    public Fish(String type, String nom, Terrain terrain, int energieMax) {
```

```
        super(type, nom, terrain);
        this.energieMax = energieMax;
        this.energie = energieMax;
    }

    protected void depenserEnergie(int cout) {
        if (!estActif()) {
            return;
        }
        energie -= cout;
        if (energie <= 0) {
            energie = 0;
            desactiverEtRetirer();
        }
    }

    protected void gagnerEnergie(int gain) {
        if (!estActif()) {
            return;
        }
        energie += gain;
        if (energie > energieMax) {
            energie = energieMax;
        }
    }
}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                   UL2IN002 - Programmation Objet                          */
/*                                                                           */
/*                        File: Movable.java                                 */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/

/**
 * Interface simple pour les objets qui peuvent se deplacer.
 */
public interface Movable {
    /**
     * Deplace l'agent d'une case vers la cible.
     *
     * @param ligneCible ligne visee
     * @param colonneCible colonne visee
     * @return true si le deplacement a eu lieu, false sinon
     * @throws SimulationException si la cible est invalide
     */
    boolean seDeplacer(int ligneCible, int colonneCible) throws SimulationException;
}
```

```
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                        File: Plankton.java                                */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/

public class Plankton extends Ressource {
    private int quantiteMax;

    public Plankton(int quantiteInitiale, int quantiteMax) {
        super("Plancton", quantiteInitiale);
        this.quantiteMax = quantiteMax;
    }

    public Plankton(Plankton autre) {
        super("Plancton", autre.getQuantite());
        this.quantiteMax = autre.quantiteMax;
    }

    public void grandir() {
        if (getQuantite() < quantiteMax) {
            setQuantite(getQuantite() + 1);
        }
    }

    public void consommer(int quantite) {
        int nouvelleQuantite = getQuantite() - quantite;
        if (nouvelleQuantite < 0) {
            nouvelleQuantite = 0;
        }
        setQuantite(nouvelleQuantite);
    }

    public boolean estEpuise() {
        return getQuantite() <= 0;
    }
}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                        File: Sardine.java                                 */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/
```

```java
import java.util.ArrayList;

/**
 * Sardine : poisson qui cherche et mange le plancton.
 */
public class Sardine extends Fish {
    public Sardine(String nom, Terrain terrain) {
        super("Sardine", nom, terrain, 6);
    }

    @Override
    public String agir(Simulation simulation) throws SimulationException {
        return agirAvecPlanktons(simulation.getPlanktons());
    }

    private String agirAvecPlanktons(ArrayList<Plankton> planktons) throws
SimulationException {
        if (!estActif()) {
            return nom + " est inactive.";
        }

        depenserEnergie(1);
        if (!estActif()) {
            return nom + " n'a plus d'energie et disparait.";
        }

        Plankton cible = chercherPlanctonLePlusProche(planktons);
        if (cible == null) {
            return nom + " ne trouve pas de plancton.";
        }

        int distance = distanceA(cible);
        int ligCible = cible.getLigne();
        int colCible = cible.getColonne();

        if (distance <= 1) {
            cible.consommer(2);
            gagnerEnergie(2);

            if (cible.estEpuise()) {
                terrain.viderCase(ligCible, colCible);
                planktons.remove(cible);
                return nom + " mange tout le plancton en (" + ligCible + "," + colCible + ").";
            }
            return nom + " mange un peu de plancton en (" + ligCible + "," + colCible + ").";
        }

        boolean deplacee = seDeplacer(ligCible, colCible);
        if (deplacee) {
```

```
            return nom + " se deplace vers le plancton.";
        }
        return nom + " est bloquee et ne peut pas avancer.";
    }

    private Plankton chercherPlanctonLePlusProche(ArrayList<Plankton> planktons) {
        Plankton meilleureCible = null;
        int meilleureDistance = Integer.MAX_VALUE;

        for (Plankton p : planktons) {
            if (p == null || p.getLigne() == -1 || p.getColonne() == -1 || p.estEpuise()) {
                continue;
            }
            int d = distanceA(p);
            if (d < meilleureDistance) {
                meilleureDistance = d;
                meilleureCible = p;
            }
        }
        return meilleureCible;
    }
}
/*******************************************************************************/
/*                                                                             */
/*                        Sorbonne Universite                                  */
/*                    UL2IN002 - Programmation Objet                           */
/*                                                                             */
/*                    File: SimulationConfig.java                              */
/*                  Etudiant: Mohamad Abobaker NASIMI                          */
/*                                                                             */
/*******************************************************************************/

/**
 * Singleton de configuration.
 */
public class SimulationConfig {
    private static SimulationConfig instance;

    private int nbLignes;
    private int nbColonnes;
    private int nombreEtapesMax;
    private int nombrePlanktons;
    private int nombreRecifs;
    private int nombreSardines;
    private int nombreTunas;
    private int quantiteInitialePlankton;
    private int quantiteMaxPlankton;

    private SimulationConfig() {
        nbLignes = 10;
```

```
    nbColonnes = 10;
    nombreEtapesMax = 20;
    nombrePlanktons = 15;
    nombreRecifs = 6;
    nombreSardines = 6;
    nombreTunas = 3;
    quantiteInitialePlankton = 3;
    quantiteMaxPlankton = 6;
}

public static SimulationConfig getInstance() {
    if (instance == null) {
        instance = new SimulationConfig();
    }
    return instance;
}

public int getNbLignes() {
    return nbLignes;
}

public int getNbColonnes() {
    return nbColonnes;
}

public int getNombreEtapesMax() {
    return nombreEtapesMax;
}

public int getNombrePlanktons() {
    return nombrePlanktons;
}

public int getNombreSardines() {
    return nombreSardines;
}

public int getNombreRecifs() {
    return nombreRecifs;
}

public int getNombreTunas() {
    return nombreTunas;
}

public int getQuantiteInitialePlankton() {
    return quantiteInitialePlankton;
}

public int getQuantiteMaxPlankton() {
```

```
        return quantiteMaxPlankton;
    }
}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                    File: SimulationException.java                         */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/

/**
 * Exception personnalisee pour la simulation.
 */
public class SimulationException extends Exception {
    public SimulationException(String message) {
        super(message);
    }

    public SimulationException(String message, Throwable cause) {
        super(message, cause);
    }
}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                        File: Simulation.java                              */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/

import java.util.ArrayList;
import java.util.Random;

/**
 * Classe principale qui gere toute la simulation.
 */
public class Simulation {
    private static final int LARGEUR_AFFICHAGE = 3;

    private Terrain terrain;
    private ArrayList<Plankton> planktons;
    private ArrayList<Ressource> recifs;
    private ArrayList<Sardine> sardines;
    private ArrayList<Tuna> tunas;
    private FishingShip fishingShip;
```

```
    private final SimulationConfig config;
    private final Random random;
    private int etapeCourante;

    public Simulation() {
        this.config = SimulationConfig.getInstance();
        this.random = new Random();
        this.etapeCourante = 0;
    }

    public void initialiser() throws SimulationException {
        creerTerrain();
        initialiserCollections();
        initialiserRessources();
        initialiserAgents();

        SimulationLogger.separateur();
        SimulationLogger.log("Simulation initialisee.");
        afficherEtat();
    }

    private void creerTerrain() throws SimulationException {
        try {
            terrain = new Terrain(config.getNbLignes(), config.getNbColonnes());
        } catch (RuntimeException e) {
            throw new SimulationException("Impossible de creer le terrain.", e);
        }
    }

    private void initialiserCollections() {
        planktons = new ArrayList<Plankton>();
        recifs = new ArrayList<Ressource>();
        sardines = new ArrayList<Sardine>();
        tunas = new ArrayList<Tuna>();
    }

    private void initialiserRessources() throws SimulationException {
        for (int i = 1; i <= config.getNombrePlanktons(); i++) {
            Plankton p = new Plankton(config.getQuantiteInitialePlankton(),
config.getQuantiteMaxPlankton());
            placerRessourceAleatoire(p);
            planktons.add(p);
        }

        for (int i = 1; i <= config.getNombreRecifs(); i++) {
            Ressource recif = new Ressource("Recif", 1);
            placerRessourceAleatoire(recif);
            recifs.add(recif);
        }
    }
```

```
    private void initialiserAgents() throws SimulationException {
        for (int i = 1; i <= config.getNombreSardines(); i++) {
            Sardine s = new Sardine("Sardine-" + i, terrain);
            placerAgentAleatoire(s);
            sardines.add(s);
        }

        for (int i = 1; i <= config.getNombreTunas(); i++) {
            Tuna t = new Tuna("Tuna-" + i, terrain);
            placerAgentAleatoire(t);
            tunas.add(t);
        }

        fishingShip = new FishingShip("Ship-1", terrain);
        placerAgentAleatoire(fishingShip);
    }

    private void placerRessourceAleatoire(Ressource ressource) throws
SimulationException {
        int maxEssais = terrain.nbLignes * terrain.nbColonnes * 3;

        for (int essai = 0; essai < maxEssais; essai++) {
            int lig = 1 + random.nextInt(terrain.nbLignes);
            int col = 1 + random.nextInt(terrain.nbColonnes);

            if (terrain.caseEstVide(lig, col)) {
                boolean ok = terrain.setCase(lig, col, ressource);
                if (ok) {
                    return;
                }
            }
        }
        throw new SimulationException("Impossible de placer " + ressource.type + " : terrain
trop rempli.");
    }

    private void placerAgentAleatoire(Agent agent) throws SimulationException {
        int maxEssais = terrain.nbLignes * terrain.nbColonnes * 3;

        for (int essai = 0; essai < maxEssais; essai++) {
            int lig = 1 + random.nextInt(terrain.nbLignes);
            int col = 1 + random.nextInt(terrain.nbColonnes);

            if (terrain.sontValides(lig, col) && !caseOccupeeParAgent(lig, col)) {
                agent.setPosition(lig, col);
                return;
            }
        }
```

```
        throw new SimulationException("Impossible de placer l'agent " + agent.getNom() +
".");
    }

    private boolean caseOccupeeParAgent(int ligne, int colonne) {
        for (Sardine s : sardines) {
            if (s != null && s.estActif() && s.getLigne() == ligne && s.getColonne() == colonne)
{
                return true;
            }
        }
        for (Tuna t : tunas) {
            if (t != null && t.estActif() && t.getLigne() == ligne && t.getColonne() == colonne) {
                return true;
            }
        }
        return fishingShip != null && fishingShip.estActif()
            && fishingShip.getLigne() == ligne && fishingShip.getColonne() == colonne;
    }

    public void executerUneEtape() throws SimulationException {
        etapeCourante++;

        SimulationLogger.separateur();
        SimulationLogger.log("ETAPE " + etapeCourante);

        faireGrandirPlanktons();
        faireAgirSardines();
        faireAgirTunas();
        faireAgirBateau();

        afficherEtat();
    }

    public void lancer() throws SimulationException {
        SimulationLogger.separateur();
        SimulationLogger.log("Debut de la simulation marine");

        for (int i = 0; i < config.getNombreEtapesMax(); i++) {
            if (tunas.isEmpty()) {
                SimulationLogger.log("Arret: il n'y a plus de thons.");
                break;
            }
            executerUneEtape();
        }

        SimulationLogger.separateur();
        SimulationLogger.log("Fin de la simulation.");
        SimulationLogger.log("Prises du bateau : " + fishingShip.getNbPrises());
    }
```

```
private void faireGrandirPlanktons() {
    int nbQuiGrandissent = 0;
    for (Plankton p : planktons) {
        if (p.getLigne() != -1 && p.getColonne() != -1) {
            int avant = p.getQuantite();
            p.grandir();
            if (p.getQuantite() > avant) {
                nbQuiGrandissent++;
            }
        }
    }
    SimulationLogger.log("Plancton: " + nbQuiGrandissent + " ressource(s) ont grandi.");
}

private void faireAgirSardines() throws SimulationException {
    if (sardines.isEmpty()) {
        SimulationLogger.log("Aucune sardine.");
        return;
    }
    ArrayList<Sardine> copie = new ArrayList<Sardine>(sardines);
    for (Sardine s : copie) {
        SimulationLogger.log(s.agir(this));
    }
    sardines.removeIf(s -> !s.estActif());
}

private void faireAgirTunas() throws SimulationException {
    if (tunas.isEmpty()) {
        SimulationLogger.log("Aucun thon.");
        return;
    }
    ArrayList<Tuna> copie = new ArrayList<Tuna>(tunas);
    for (Tuna t : copie) {
        SimulationLogger.log(t.agir(this));
    }
    tunas.removeIf(t -> !t.estActif());
}

private void faireAgirBateau() throws SimulationException {
    SimulationLogger.log(fishingShip.agir(this));
}

private void afficherEtat() {
    afficherResume();
    afficherPositionsAgents();
    afficherTerrainComplet();
}

private void afficherResume() {
```

```
        SimulationLogger.log(
            "Resume: planctons=" + planktons.size() +
            ", recifs=" + recifs.size() +
            ", sardines=" + sardines.size() +
            ", tunas=" + tunas.size()
        );
    }

    private void afficherPositionsAgents() {
        for (Sardine s : sardines) {
            SimulationLogger.log(s.getNom() + " en (" + s.getLigne() + "," + s.getColonne() +
")");
        }
        for (Tuna t : tunas) {
            SimulationLogger.log(t.getNom() + " en (" + t.getLigne() + "," + t.getColonne() +
")");
        }
        if (fishingShip != null) {
            SimulationLogger.log(fishingShip.getNom() + " en (" + fishingShip.getLigne() + "," +
fishingShip.getColonne() + ")");
        }
    }

    private void afficherTerrainComplet() {
        String[][] vue = construireVueBase();
        poserSardines(vue);
        poserTunas(vue);
        poserSymboleAgent(vue, fishingShip, "Shi");
        imprimerVue(vue, terrain.nbLignes, terrain.nbColonnes);
    }

    private String[][] construireVueBase() {
        String[][] vue = new String[terrain.nbLignes][terrain.nbColonnes];

        for (int lig = 1; lig <= terrain.nbLignes; lig++) {
            for (int col = 1; col <= terrain.nbColonnes; col++) {
                Ressource ressource = terrain.getCase(lig, col);
                if (ressource == null) {
                    vue[lig - 1][col - 1] = "";
                } else {
                    vue[lig - 1][col - 1] = symboleRessource(ressource);
                }
            }
        }
        return vue;
    }

    private void poserSardines(String[][] vue) {
        for (Sardine s : sardines) {
            poserSymboleAgent(vue, s, "Sar");
```

```
    }
}

private void poserTunas(String[][] vue) {
    for (Tuna t : tunas) {
        poserSymboleAgent(vue, t, "Tun");
    }
}

private void poserSymboleAgent(String[][] vue, Agent agent, String symbole) {
    if (agent == null || !agent.estActif() || !agent.estPlace()) {
        return;
    }

    int lig = agent.getLigne();
    int col = agent.getColonne();
    if (!terrain.sontValides(lig, col)) {
        return;
    }

    vue[lig - 1][col - 1] = symbole;
}

private String symboleRessource(Ressource ressource) {
    String type = ressource.type;
    if ("Plancton".equalsIgnoreCase(type)) {
        return "Pla";
    }
    if ("Recif".equalsIgnoreCase(type)) {
        return "Rec";
    }
    return premiersCaracteres(type, 3);
}

private String premiersCaracteres(String texte, int tailleMax) {
    if (texte == null || texte.isEmpty()) {
        return "";
    }
    if (texte.length() <= tailleMax) {
        return texte;
    }
    return texte.substring(0, tailleMax);
}

private void imprimerVue(String[][] vue, int nbLignes, int nbColonnes) {
    int largeurCase = LARGEUR_AFFICHAGE;
    for (int lig = 0; lig < nbLignes; lig++) {
        for (int col = 0; col < nbColonnes; col++) {
            largeurCase = Math.max(largeurCase, vue[lig][col].length());
        }
```

```
        }

        String separateur = construireSeparateur(largeurCase, nbColonnes);
        StringBuilder affichage = new StringBuilder();
        affichage.append(separateur).append("\n");

        for (int lig = 0; lig < nbLignes; lig++) {
            for (int col = 0; col < nbColonnes; col++) {
                affichage.append("|");
                affichage.append(String.format("%-" + largeurCase + "s", vue[lig][col]));
            }
            affichage.append("|\n");
            affichage.append(separateur).append("\n");
        }

        System.out.print(affichage.toString());
    }

    private String construireSeparateur(int largeurCase, int nbColonnes) {
        StringBuilder tirets = new StringBuilder();
        for (int i = 0; i < largeurCase; i++) {
            tirets.append("-");
        }

        StringBuilder separateur = new StringBuilder(":");
        for (int col = 0; col < nbColonnes; col++) {
            separateur.append(tirets).append(":");
        }
        return separateur.toString();
    }

    public ArrayList<Plankton> getPlanktons() {
        return planktons;
    }

    public ArrayList<Sardine> getSardines() {
        return sardines;
    }

    public ArrayList<Tuna> getTunas() {
        return tunas;
    }
}
/*******************************************************************************/
/*                                                                             */
/*                        Sorbonne Universite                                  */
/*                    UL2IN002 - Programmation Objet                           */
/*                                                                             */
/*                    File: SimulationLogger.java                              */
/*                  Etudiant: Mohamad Abobaker NASIMI                          */
```

```
/*                                                                      */
/***************************************************************************/

public final class SimulationLogger {
    private SimulationLogger() {
    }

    public static void log(String message) {
        System.out.println(message);
    }

    public static void separateur() {
        log("-----------------------------------------------");
    }
}
/***************************************************************************/
/*                                                                      */
/*                        Sorbonne Universite                        */
/*                    UL2IN002 - Programmation Objet                    */
/*                                                                      */
/*                      File: TestSimulation.java                      */
/*                   Etudiant: Mohamad Abobaker NASIMI                   */
/*                                                                      */
/***************************************************************************/

/**
 * Point d'entree du projet.
 */
public class TestSimulation {
    public static void main(String[] args) {
        try {
            Simulation simulation = new Simulation();
            simulation.initialiser();
            simulation.lancer();
        } catch (SimulationException e) {
            System.out.println("Erreur de simulation: " + e.getMessage());
        }
    }
}
/***************************************************************************/
/*                                                                      */
/*                        Sorbonne Universite                        */
/*                    UL2IN002 - Programmation Objet                    */
/*                                                                      */
/*                      File: TestTerrain.java                       */
/*                   Etudiant: Mohamad Abobaker NASIMI                   */
/*                                                                      */
/***************************************************************************/

import java.util.ArrayList;
```

```
/**
 * Exemple d'utilisation des classes <code>Terrain</code>
 * et <code>Ressource</code>.
 */
public class TestTerrain {
	public static void main(String[] args) {

		// Exemple de création de terrain
		Terrain t = new Terrain(3,7);

		// Terrain initial : il est vide
		t.afficher(2);

		// Informations sur le terrain
		System.out.println(t);

		// On créé une ressource...
		Ressource ress1 = new Ressource("Thon",5);
		System.out.println("\nCréation de ress1="+ress1);

		// ... et on la place sur le terrain
		if (t.setCase(2,3,ress1))
			System.out.println("Ajout de " +ress1+" effectuée !");
		else
			System.out.println("Ajout de " +ress1+" impossible, la case est
occupée !");

		// On créé une autre ressource...
		Ressource ress2 = new Ressource("Requin",4);
		System.out.println("\nCréation de ress2="+ress2);

		// ... et on essaie de la placer à la même place
		if (t.setCase(2,3,ress2))
			System.out.println("Ajout de " +ress2+" effectuée !");
		else
			System.out.println("Ajout de " +ress2+" impossible, la case est
occupée !");

		// On essaie de placer la ressource à un autre endroit
		if (t.setCase(1,4,ress2))
			System.out.println("Ajout de " +ress2+" effectuée !");
		else
			System.out.println("Ajout de " +ress2+" impossible, la case est
occupée !");

		// Ajout d'une troisième ressource
		if (t.caseEstVide(3,5)) {
			t.setCase(3,5,new Ressource("Raie",3));
		}
```

```
		System.out.println();

		// Affichage du terrain avec les ressources ajoutées
		t.afficher(6);

		// Informations sur le terrain
		System.out.println(t);

		System.out.println();
		System.out.println("Contenu d'une case :");
		System.out.println("- Contenu de la case (2,3) : "+t.getCase(2,3));
		System.out.println("- Contenu de la case (1,4) : "+t.getCase(1,4));
		System.out.println("- Contenu de la case (1,1) : "+t.getCase(1,1));

		System.out.println();
		System.out.println("Vidage d'une case :");
		System.out.println("- Vidage de la case (1,4) : "+t.viderCase(1,4));
		System.out.println("- Vidage de la case (1,1) : "+t.viderCase(1,1));

		System.out.println();

		// Affichage du terrain
		t.afficher(4);

		System.out.println("Liste des ressources présentes actuellement sur le
terrain :");
		ArrayList<Ressource> liste = t.lesRessources();
		for (Ressource r : liste) {
			System.out.println("- "+r);
		}

	}

}
/*****************************************************************************/
/*                                                                           */
/*                        Sorbonne Universite                                */
/*                    UL2IN002 - Programmation Objet                         */
/*                                                                           */
/*                         File: Tuna.java                                   */
/*                  Etudiant: Mohamad Abobaker NASIMI                        */
/*                                                                           */
/*****************************************************************************/

import java.util.ArrayList;

/**
 * Tuna : poisson predateur qui chasse les sardines.
 */
```

```
public class Tuna extends Fish {
    public Tuna(String nom, Terrain terrain) {
        super("Tuna", nom, terrain, 8);
    }

    @Override
    public String agir(Simulation simulation) throws SimulationException {
        return agirAvecSardines(simulation.getSardines());
    }

    private String agirAvecSardines(ArrayList<Sardine> sardines) throws
SimulationException {
        if (!estActif()) {
            return nom + " est inactif.";
        }

        depenserEnergie(1);
        if (!estActif()) {
            return nom + " n'a plus d'energie et disparait.";
        }

        Sardine cible = chercherSardineLaPlusProche(sardines);
        if (cible == null) {
            return nom + " ne trouve pas de sardine.";
        }

        int distance = distanceA(cible);
        int ligCible = cible.getLigne();
        int colCible = cible.getColonne();

        if (distance <= 1) {
            cible.desactiverEtRetirer();
            sardines.remove(cible);
            gagnerEnergie(3);

            seDeplacer(ligCible, colCible);
            return nom + " mange " + cible.getNom() + ".";
        }

        boolean deplace = seDeplacer(ligCible, colCible);
        if (deplace) {
            return nom + " se deplace vers une sardine.";
        }
        return nom + " est bloque.";
    }

    private Sardine chercherSardineLaPlusProche(ArrayList<Sardine> sardines) {
        Sardine meilleureCible = null;
        int meilleureDistance = Integer.MAX_VALUE;
```

```
        for (Sardine s : sardines) {
            if (s == null || !s.estActif() || s.getLigne() == -1 || s.getColonne() == -1) {
                continue;
            }
            int d = distanceA(s);
            if (d < meilleureDistance) {
                meilleureDistance = d;
                meilleureCible = s;
            }
        }
        return meilleureCible;
    }
}
```